

NOVAC Internship – Task 2 Daily Documentation

Task: File Upload APIs, MongoDB Integration, Semantic Chunking, and Embedding Storage

Overview

Today's work focused on transforming the existing FastAPI document upload prototype into a more advanced AI-powered document management and preprocessing system. The implementation included user information handling, PDF upload support, semantic chunking with token limits, editable frontend chunk visualization, and MongoDB update operations.

The system now allows users to enter their name and email, upload TXT or PDF documents, automatically extract and process text using NLP techniques, generate embeddings, store chunks inside MongoDB, display them dynamically on the frontend, and edit chunk contents directly from the user interface with real-time database updates.

Resources and References Used

- FastAPI Official Documentation
- MongoDB Documentation
- MongoDB Compass
- Hugging Face Sentence Transformers
- NLTK Documentation
- PyPDF Documentation
- scikit-learn Documentation

Concepts Learned

Embeddings: Embeddings are vector representations of text generated using AI models. They convert human-readable language into numerical vectors that preserve semantic meaning.

Vector Similarity: Learned how cosine similarity is used to compare embeddings and determine semantic closeness between sentences.

Semantic Chunking: Implemented semantic chunking where related sentences are grouped together based on similarity instead of fixed-size splitting.

Token Limit-Based Chunking: Understood why chunk size limits are important for Large Language Models (LLMs). Implemented chunk control using approximate token limits based on word count thresholds.

Natural Language Processing (NLP): Used NLP preprocessing techniques such as sentence tokenization and semantic grouping for document understanding.

PDF Text Extraction: Implemented PDF parsing using the pypdf library to extract readable text from uploaded PDF files.

MongoDB CRUD Operations:

Learned and implemented:

- insert_one() for storing chunks
- update_one() for updating edited chunk data
- MongoDB document storage structure

Editable Frontend Interfaces: Created editable chunk textareas using HTML and JavaScript so users can modify chunk content directly from the frontend.

FastAPI Form Handling: Used FastAPI Form() and UploadFile() handling for accepting user details and file uploads together.

Libraries, Modules, and Components Learned

FastAPI: Used for building asynchronous backend APIs.

UploadFile and File: Used for handling uploaded files efficiently in FastAPI.

Form: Used for accepting name and email form inputs.

SentenceTransformer: Used for generating semantic embeddings using the all-MiniLM-L6-v2 model.

cosine_similarity: Used to calculate semantic similarity between sentence embeddings.

nltk.sent_tokenize: Used for sentence tokenization.

pymongo: Used for MongoDB database communication.

pypdf: Used for PDF text extraction.

BaseModel (Pydantic): Used for validating chunk update request data.

Practical Implementation

User Input System: Added frontend fields for:

- Name
- Email

PDF and TXT Upload System: Implemented support for:

- .txt
- .pdf

Semantic Chunking Pipeline: Implemented:

- sentence tokenization
- semantic similarity comparison
- token-limited chunk grouping

Chunks are now generated based on both:

- semantic meaning
- maximum word/token thresholds

Embedding Generation: Generated embeddings for each chunk using the Hugging Face Sentence Transformer model.

MongoDB Storage: Stored the following data in MongoDB:

- user name
- user email
- filename
- chunk ID
- chunk text
- embedding vectors

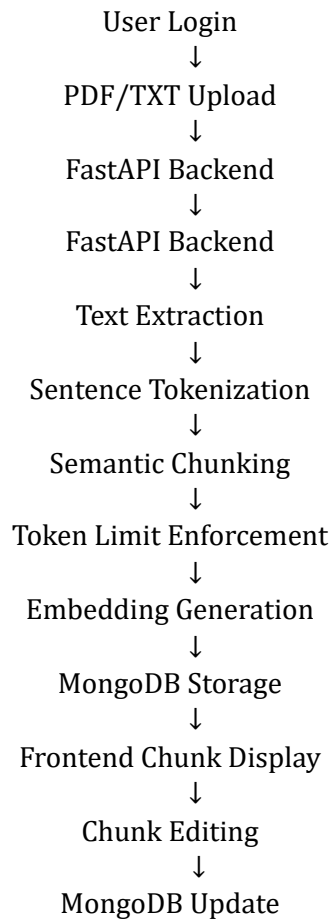
Editable Frontend Chunk Display: Displayed chunks dynamically in the frontend using:

- HTML
- JavaScript
- textareas

MongoDB Chunk Updating: Implemented a new FastAPI route:

- /update-chunk

Architecture Understood



Challenges Faced

Several implementation challenges were encountered during development:

- Managing sentence grouping while maintaining token limits.
- Updating MongoDB documents dynamically after frontend edits.
- Handling different document formats (TXT and PDF).
- Understanding semantic chunking workflows.
- Managing embedding generation for both sentences and final chunks.
- Designing editable frontend interfaces connected to backend APIs.

These challenges helped improve understanding of AI preprocessing pipelines, backend architecture, and full-stack API integration.

Conclusion

Today's implementation significantly upgraded the previous prototype into a more complete AI document ingestion and management system. The application can now process user-uploaded documents, generate semantic chunks with token control, create embeddings, store vectorized data inside MongoDB, display chunks dynamically on the frontend, and allow live editing with database synchronization.

The concepts implemented today form the foundation of modern Retrieval-Augmented Generation (RAG) systems, vector databases, semantic retrieval systems, and AI-powered document processing workflows.

Prepared By

Divesh Kumar S